Міністерство освіти і науки України
Вінницький національний технічний університет
Факультет інформаційних технологій та комп’ютерної інженерії
Кафедра ПЗ

Практична робота №4

з дисципліни «Розробка проєктів засобами платформи .NET»

Виконав: ст. 1ПІ-24Б Кузя Н.І.
Перевірив: викл. Позур М.Ю.

м. Вінниця – 2026

Тема: Розробка проєктів засобами платформи .NET

Тема: LINQ to Objects та асинхронне програмування

Варіант: 4 (Відстеження посилок)

1. Короткий опис реалізованого функціоналу

У даній роботі розроблено консольний застосунок для відстеження посилок, що використовує асинхронні операції та запити LINQ. Структури даних описані за допомогою record (Parcel, Order). Реалізовано три асинхронні джерела даних, які імітують звернення до API різних служб доставки через Task.Delay. Застосунок має функцію як послідовного завантаження даних (await по черзі), так і паралельного (Task.WhenAll), час виконання яких вимірюється класом Stopwatch. Аналіз зібраних даних виконується за допомогою запитів LINQ (Where, GroupBy, Join/GroupJoin, OrderBy). Звіт щодо статусу відправлень асинхронно зберігається у файл формату JSON (з використанням JsonSerializer) з урахуванням обробки винятків (try-catch на випадок IOException).

2. Посилання на репозиторій GitHub

https://github.com/shwballl/vntu-dotnet

3. Лістинг коду (Program.cs)

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.IO;
using System.Linq;
using System.Text.Encodings.Web;
using System.Text.Json;
using System.Text.Unicode;
using System.Threading.Tasks;

// Дані (Records)
public record Parcel(string TrackingNumber, string Carrier, string City, string Status,
double WeightKg, int DaysInTransit);
public record Order(string OrderId, string Customer, string TrackingNumber);
```

```
class Program
{
    // Замовлення (зберігаються в пам'яті)
    static readonly List<Order> _orders = new()
    {
        new("ORD-01", "Олександр", "UA-101"),
        new("ORD-02", "Марія", "UA-102"),
        new("ORD-03", "Іван", "UA-103"),
        new("ORD-04", "Олена", "UA-201"),
        new("ORD-05", "Петро", "UA-202"),
        new("ORD-06", "Світлана", "UA-203"),
        new("ORD-07", "Ганна", "UA-301"),
        new("ORD-08", "Дмитро", "UA-302"),
        new("ORD-09", "Олександр", "UA-303"),
        new("ORD-10", "Наталія", "UA-999") // Граничний випадок (номера немає в жодній службі)
    };

    static List<Parcel> _parcels = new();

    static readonly JsonSerializerOptions _jsonOptions = new()
    {
        WriteIndented = true,
        Encoder = JavaScriptEncoder.Create(UnicodeRanges.All)
    };

    static async Task Main()
    {
        bool isRunning = true;
        while (isRunning)
        {
```

```
        Console.WriteLine("\n=== Меню (Варіант 4. Відстеження посилок) ===");
        Console.WriteLine("0. Вихід");
        Console.WriteLine("1. Завантажити дані (одночасно)");
        Console.WriteLine("2. Завантажити дані (послідовно)");
        Console.WriteLine("3. Статистика за службами");
        Console.WriteLine("4. Середній термін доставки отриманих посилок");
        Console.WriteLine("5. Затримані посилки (В дорозі довше N днів)");
        Console.WriteLine("6. Замовлення клієнта");
        Console.WriteLine("7. Зберегти звіт у файл");

        Console.Write("Ваш вибір: ");
        string? choice = Console.ReadLine();

        switch (choice)
        {
            case "1": await HandleLoadAsync(parallel: true); break;
            case "2": await HandleLoadAsync(parallel: false); break;
            case "3": HandleStatsByCarrier(); break;
            case "4": HandleAverageDeliveryTime(); break;
            case "5": HandleDelayedParcels(); break;
            case "6": HandleCustomerOrders(); break;
            case "7": await HandleSaveReportAsync(); break;
            case "0": isRunning = false; break;
            default: Console.WriteLine("Некоректний вибір."); break;
        }
    }
}

// === Джерела даних (імітація затримок) ===
static async Task<List<Parcel>> LoadShvydkoPoshtaAsync()
{
    await Task.Delay(2000);
```

```
    return new List<Parcel>
    {
        new("UA-101", "ШвидкоПошта", "Київ", "В дорозі", 2.5, 3),
        new("UA-102", "ШвидкоПошта", "Львів", "Отримано", 1.2, 2),
        new("UA-103", "ШвидкоПошта", "Одеса", "У відділенні", 5.0, 4)
    };
}

static async Task<List<Parcel>> LoadExpressDostavkaAsync()
{
    await Task.Delay(1500);
    return new List<Parcel>
    {
        new("UA-201", "ЕкспресДоставка", "Харків", "В дорозі", 3.0, 6),
        new("UA-202", "ЕкспресДоставка", "Дніпро", "В дорозі", 1.5, 7),
        new("UA-203", "ЕкспресДоставка", "Запоріжжя", "Отримано", 0.5, 1)
    };
}

static async Task<List<Parcel>> LoadPosylka24Async()
{
    await Task.Delay(1000);
    return new List<Parcel>
    {
        new("UA-301", "Посилка24", "Вінниця", "У відділенні", 4.2, 5),
        new("UA-302", "Посилка24", "Житомир", "Отримано", 2.1, 3),
        new("UA-303", "Посилка24", "Тернопіль", "В дорозі", 1.8, 8)
    };
}

// === Обробники пунктів меню ===
static async Task HandleLoadAsync(bool parallel)
```

```
    {
        Console.WriteLine("Завантаження даних...");
        var sw = Stopwatch.StartNew();
        List<Parcel>[] parts;

        if (parallel)
        {
            parts = await Task.WhenAll(LoadShvydkoPoshtaAsync(),
LoadExpressDostavkaAsync(), LoadPosylka24Async());
        }
        else
        {
            parts = new[] { await LoadShvydkoPoshtaAsync(), await
LoadExpressDostavkaAsync(), await LoadPosylka24Async() };
        }

        sw.Stop();
        _parcels = parts.SelectMany(p => p).ToList();
        Console.WriteLine($"Завантажено {_parcels.Count} посилок за
{sw.ElapsedMilliseconds} мс.");
    }

    static void HandleStatsByCarrier()
    {
        if (!HasData()) return;

        var stats = _parcels
            .GroupBy(p => new { p.Carrier, p.Status })
            .Select(g => new { Carrier = g.Key.Carrier, Status = g.Key.Status, Count =
g.Count(), TotalWeight = g.Sum(p => p.WeightKg) })
            .OrderBy(s => s.Carrier)
            .ThenBy(s => s.Status);
```

```
        foreach (var s in stats)
        {
            Console.WriteLine($"{s.Carrier} | Статус: {s.Status} | Кількість: {s.Count} |
Загальна вага: {s.TotalWeight:F1} кг");
        }
    }

    static void HandleAverageDeliveryTime()
    {
        if (!HasData()) return;

        var avgTimeQuery =
            from p in _parcels
            where p.Status == "Отримано"
            group p by p.Carrier into g
            orderby g.Average(x => x.DaysInTransit)
            select new { Carrier = g.Key, AvgDays = g.Average(x => x.DaysInTransit) };

        Console.WriteLine("Середній термін доставки отриманих посилок:");
        foreach (var item in avgTimeQuery)
        {
            Console.WriteLine($"{item.Carrier}: {item.AvgDays:F1} днів");
        }
    }

    static void HandleDelayedParcels()
    {
        if (!HasData()) return;

        Console.Write("Введіть кількість днів (N): ");
        if (int.TryParse(Console.ReadLine(), out int n) && n > 0)
```

```
        {
            var delayed = _parcels
                .Where(p => p.Status == "В дорозі" && p.DaysInTransit > n)
                .OrderByDescending(p => p.DaysInTransit)
                .ThenBy(p => p.Carrier);

            if (!delayed.Any())
            {
                Console.WriteLine("Затриманих посилок не знайдено.");
                return;
            }

            foreach (var d in delayed)
            {
                Console.WriteLine($"Номер: {d.TrackingNumber} | Служба: {d.Carrier} |
Місто: {d.City} | Днів в дорозі: {d.DaysInTransit}");
            }
        }
        else
        {
            Console.WriteLine("Некоректне значення днів.");
        }
    }

    static void HandleCustomerOrders()
    {
        if (!HasData()) return;

        Console.Write("Введіть ім'я клієнта: ");
        string? name = Console.ReadLine();

        // Групове з'єднання (LEFT JOIN) для обробки неіснуючих номерів
```

```
        var customerOrders = _orders
            .Where(o => o.Customer.Equals(name, StringComparison.OrdinalIgnoreCase))
            .GroupJoin(_parcels,
                o => o.TrackingNumber,
                p => p.TrackingNumber,
                (order, parcelGroup) => new
                {
                    order.OrderId,
                    order.TrackingNumber,
                    ParcelInfo = parcelGroup.FirstOrDefault()
                });

        if (!customerOrders.Any())
        {
            Console.WriteLine("Замовлень не знайдено.");
            return;
        }

        foreach (var co in customerOrders)
        {
            string status = co.ParcelInfo?.Status ?? "статус невідомий";
            string city = co.ParcelInfo?.City ?? "невідомо";
            Console.WriteLine($"Замовлення: {co.OrderId} | Номер відправлення:
{co.TrackingNumber} | Статус: {status} | Місто: {city}");
        }
    }

    static async Task HandleSaveReportAsync()
    {
        var report = _orders
            .GroupJoin(_parcels,
                o => o.TrackingNumber,
```

```
            p => p.TrackingNumber,
            (order, parcelGroup) => new
            {
                Order = order.OrderId,
                Customer = order.Customer,
                TrackingNumber = order.TrackingNumber,
                Status = parcelGroup.FirstOrDefault()?.Status ?? "статус невідомий"
            })
        .ToList();

    string json = JsonSerializer.Serialize(report, _jsonOptions);

    try
    {
        await File.WriteAllTextAsync("report.json", json);
        Console.WriteLine("Звіт успішно збережено у файл report.json");
    }
    catch (IOException ex)
    {
        Console.WriteLine($"Не вдалося зберегти звіт: {ex.Message}");
    }
}

static bool HasData()
{
    if (_parcels.Count > 0) return true;
    Console.WriteLine("Немає даних. Спочатку завантажте їх (пункт 1 або 2).");
    return false;
}
}
```

Висновок: під час виконання роботи було засвоєно механізми асинхронного програмування в C#. Практично доведено, що використання Task.WhenAll для незалежних I/O-операцій суттєво зменшує загальний час очікування порівняно з послідовним використанням await. Також було застосовано потужні можливості LINQ to Objects для аналізу та агрегації колекцій (зокрема метод GroupJoin, який дозволив обробити граничний випадок із замовленнями, відправлення яких не знайдені в службах доставки). Опановано базові навички серіалізації даних у JSON.